

Name: Meseret Getachew
ID: BITS/UGR/0144/24
Section: 02

Personal Software Process (PSP)

Part 2(ii) Student grade management system

1. Introduction

For this project, I worked on a simple Student Grade Management System using Python. The program allows a user to enter a student's name, subject names, and marks. It checks whether the marks are valid, calculates the average mark, assigns a letter grade and displays the final result.

To complete the project, I divided the work into five jobs: requirements analysis, design, implementation, testing, and documentation. Before starting each job, I estimated the time needed and the expected work product. After completing each job, I recorded the actual time and completed work products in the Job Number Log.

2. Identifying Jobs and Making Estimates

Before starting the project, I identified the main jobs and estimated how long each one would take. I also identified the work product expected from each job.

Job No.	Job Description	Estimated Time	Expected Work Product	Unit
1	Requirements Analysis	45 minutes	Requirements document	1 document
2	Design	50 minutes	Program flowchart	1 flowchart
3	Implementation	2 hours 20 minutes	Python source code	1 program
4	Testing	1 hour 15 minutes	Test cases	5 test cases
5	Documentation	50 minutes	Project report	1 report
Total		**6 hours**		**9 units**

The total estimated working time was six hours. The nine work-product units include one requirements document, one flowchart, one program, five test cases, and one report.

3. Daily Time Log

I used the Daily Time Log to record the actual working time and interruptions during the project. The following is the assumed record for this assignment.

Job No.	Job Description	Actual Working Time	Interruption Time
1	Requirements Analysis	1 hr 10 min	0 min
2	Design	1 hr 15 min	0 min
3	Implementation	2 hr 50 min	12 min
4	Testing	1 hr 40 min	38 min
5	Documentation	1 hr 15 min	7 min
Total		**8 hr 10 min**	**57 min**

Interruption Record

During the project, I experienced the following interruptions:

Interruption	Time Lost
Answering a phone call	12 minutes
Taking a snack break	8 minutes
Doomscrolling on my phone	30 minutes
Zoning out and losing focus	7 minutes
Total	**57 minutes**

The total actual working time was 8 hours and 10 minutes. Including the interruptions, the total elapsed time was 9 hours and 7 minutes.

4. Job Number Log After Completing Each Job

After completing each job, I recorded the actual time spent and the number of work products completed. I also updated the cumulative data to show the total time and units from the first job up to the current job.

Job No.	Job Description	Estimated Time	Actual Time	Difference	Estimated Units	Actual Units	Cumulative Actual Time

1	Requirements Analysis	45 min	1 hr 10 min	+25 min	1	1	1 hr 10 min
2	Design	50 min	1 hr 15 min	+25 min	1	1	2 hr 25 min
3	Implementation	2 hr 20 min	2 hr 50 min	+30 min	1	1	5 hr 15 min
4	Testing	1 hr 15 min	1 hr 40 min	+25 min	5	5	6 hr 55 min
5	Documentation	50 min	1 hr 15 min	+25 min	1	1	8 hr 10 min
Total		**6 hr**	**8 hr 10 min**	**+2 hr 10 min**	**9**	**9**	**8 hr 10 min**

The estimated time was six hours, but the actual working time was eight hours and ten minutes. Each job took at least 25 minutes longer than estimated. However, all the expected work products were completed.

5. Cumulative Work-Product Units

The cumulative units show the total number of work products completed as the project progressed.

Job No.	Job Description	Actual Units Completed	Cumulative Units
1	Requirements Analysis	1	1
2	Design	1	2
3	Implementation	1	3
4	Testing	5	8
5	Documentation	1	9
Total		**9**	**9**

Short Notes on PSP, TSP, CMMI, Product Planning, Size Measurement, and Commitment

1. PSP, TSP, and CMMI

PSP (Personal Software Process): PSP is a process that helps individual software developers improve their personal performance, manage their time, estimate their work and reduce errors.

TSP (Team Software Process): TSP is a process that helps software development teams plan, organize, manage and monitor their work to achieve project goals and deliver quality software.

CMMI (Capability Maturity Model Integration): CMMI is a framework that helps organizations improve their software development processes, maintain quality, manage projects and achieve better results.

Elements of TSP

1. Team Planning: Setting project goals, estimating time and effort and dividing tasks among team members.
2. Defined Roles: Giving each team member clear responsibilities.
3. Progress Tracking: Monitoring the team's work to make sure the project is on schedule.
4. Risk Management: Identifying possible problems and preparing solutions.
5. Continuous Improvement: Learning from previous work and finding ways to improve team performance.

Relationship Between PSP and TSP

PSP and TSP complement each other because PSP focuses on improving the performance of individual developers, while TSP focuses on improving the performance of the entire team. When individuals manage their work effectively, the whole team can perform better.

Advantages and Disadvantages

PSP

Advantages:

- Encourages personal discipline.
- Helps estimate time and effort.
- Helps identify personal error patterns and improve performance.

Disadvantages:

- Requires consistent personal data collection.

- Can feel time-consuming for beginners.

TSP

Advantages:

- Encourages teamwork and coordination.
- Helps teams manage projects effectively.
- Improves task distribution and project monitoring.

Disadvantages:

- Requires good communication and team commitment.
- Can involve additional planning and meetings.

2. Product Planning Process

Product planning is the process of deciding what needs to be developed, how it will be developed and what resources are required.

Important points about product planning:

- Product planning is not an exact process because estimates and actual results may differ.
- When preparing a product plan, we should compare what we plan to do with what we have actually done.
- We should start by collecting data from similar previous jobs or projects.
- We then compare the new task with previous tasks to make better estimates of time, effort and resources.

3. Size Measurement (Lines of Code)

Size measurement is the process of estimating how large a software product is.

One common method is Lines of Code (LOC) which measures software size by counting the number of lines written in a programming language.

Importance of LOC:

- Helps estimate software size.
- Helps estimate development effort and time.
- Helps compare the size of different software projects.

However, LOC can be difficult to estimate before coding and may vary depending on the programming language.

4. Commitment

Commitment in software engineering means voluntarily agreeing to take responsibility for completing a task or achieving a specific goal.

Key Characteristics of Commitment

1. Voluntary: Commitment should be made willingly, not through force or pressure.
2. True Agreement: A genuine agreement is the most important characteristic of personal commitment.
3. Communication: If we realize that we cannot fulfill a commitment, we should inform the person we made the commitment to as early as possible.
4. Responsibility: We should make realistic promises and do our best to fulfill them.

Importance of Commitment

The most important single asset a software engineer can have is a reputation for meeting commitments. This means people should be able to trust a software engineer to keep promises, complete assigned work and deliver what they have agreed to do.